

# 操作系统 第4部分 存储管理

## 2、虚拟存储器基本理论

**惰性内存分配：利用程序运行局部性原理。分配物理页框，延迟至逻辑页第一次访问时刻。**

---

同济大学计算机系

# 1、程序运行的局部性原理

- **程序的局部性原理是指程序在执行时呈现出局部性规律，即在一段时间内，整个程序的执行仅限于程序中的某一部分。**
- **局部性原理表现为：时间局部性和空间局部性。**
  - **时间局部性：当前执行的指令，不久后可能再次执行；现在正在访问的数据，过会还会访问。** 因为，程序里有循环。
  - **空间局部性：与当前访问的指令/数据相邻的存储单元，不久后被访问的概率非常高。** 因为，指令通常是顺序存放、顺序执行的。很多数据结构，如数组、向量、表，元素也是存放在一起的。

# 传统存储管理方式，一次性为进程图像分配所需的全部内存

完全没必要。原因是很多程序分支根本执行不到

- 会降低进程图像加载、换入、换出速度。
- 会降低物理内存的利用率。减少内存中并发进程的数量。降低系统算力。

# 改善性能的思路

- 仅为当前需要访问的代码和数据分配内存。
- 现在主流的方法：在基本的分页机制的基础上，加上调页机制 和 页面置换机制
  - 调页机制：为即将访问的页面分配存储空间、并且进行初始化。
    - 请求调页：内存分配（调页操作），延迟至页面访问时刻。一次调入一页。
    - 预调页：一次调入多个页面。当前需要的页面 + 虚空间中附近的多个页面。
  - 页面置换机制：淘汰暂时不用的页面。将其换至磁盘，释放内存。
- 基于此技术打造的虚拟存储器，运行 时间 & 空间局部性 表现良好的程序，速度不亚于全部图像驻留物理内存。

## 2、虚拟存储器

- 1961年英国曼彻斯特大学提出虚拟存储器的概念。虚拟存储器用硬盘中的swap分区（或者swap文件）作为主存的**辅助存储器**，存放所有进程的图像；主存中只存放CPU最近需要使用的逻辑页面。
- 有了虚拟存储器，只需要装入少量逻辑页面进程就可以运行，访问到其它逻辑页面时，MMU会抛出异常，**缺页异常处理程序**调入所缺页面。
- 虚拟存储器的优点在于：
  - 启动前不必装入全部进程图像，应用程序装载（exec）速度快。
  - 程序员编程不必受物理内存的限制，计算机可以运行虚空间大于内存总容量的应用程序。
  - 最后，除非内存抖动（所有进程均内存不足），系统不再需要执行swap in 和swap out操作。

### 3、请求调页

- 任何应用程序，只需为初始栈分配主存页框，装入main函数栈帧后便可以开始运行。其余进程图像，一次一页，访问时，缺页中断处理程序负责调入。
- 请求调页是指如果CPU访问的数据或指令不在主存，引发缺页异常，异常处理程序调入所缺逻辑页面。

### 4、页面置换（for 请求调页系统）

- 缺页时，异常处理程序会为现运行进程分配一个可用的主存页框。如果内存没了，就要覆盖一个逻辑页面，用分配给它的物理页框装新的（所缺）逻辑页。这个操作就是页面置换。

## 3.1 缺页 (page fault)

- 缺页：CPU访问的信息内存中不存在。以i386芯片为例，present bit是0。
- 缺页异常处理过程：
  - **MMU硬件**(1)当前指令执行时操作数不在内存(2)取指时下条指令不在内存时抛出异常
  - **中断处理元件** (1) 压栈引发异常的当前指令的地址 (2) cr2记录引发缺页的那个逻辑地址 (3) 其余硬件中断响应流程，激活内核
  - **内核**执行缺页异常处理程序 (1) 找1个空闲的物理页框 (2) 在页表中定位目标逻辑页对应的PTE，修改它填入pageFrame, present bit置1 (3) 初始化物理页框
  - 异常返回 (**IRET**)，进程重新执行引发异常的指令。所缺数据在内存，不会再次引发缺页异常，程序得以继续运行。



## 4.1 页面置换

- 内存不足时，覆盖一个逻辑页。
- 页面置换方式（1）固定分配局部置换（2）可变分配局部置换（3）可变分配全局置换
  - 固定 vs 可变                      系统为进程分配的物理页框集合不变。
  - 局部 vs 全局                    被覆盖的只能是进程自己的逻辑页。

## 固定分配局部置换

- 为每个进程分配固定数目的物理页框。如果缺页，只能覆盖自己的逻辑页面。
  - 典型算法：FIFO、LRU、clock...

## 可变分配局部置换

- 为每个进程分配一定数目的物理页框。如果缺页，依然只能覆盖自己的逻辑页面。
- 跟踪进程的执行情况
  - 频繁缺页，为进程追加物理页框。
  - 缺页率长期维持很低的水平，可以回收部分页框。

## 可变分配全局置换

- 为每个进程分配一定数目的物理块（就是物理页框）
- 系统维护一个全局空闲物理块队列
- 当某进程发生缺页时，从空闲物理块队列中取出一个分配给该进程。
- 空闲队列分配完时，OS才会覆盖内存中的逻辑页面。
- 由于被覆盖的可能是其它进程的逻辑页面。所以是全局置换。



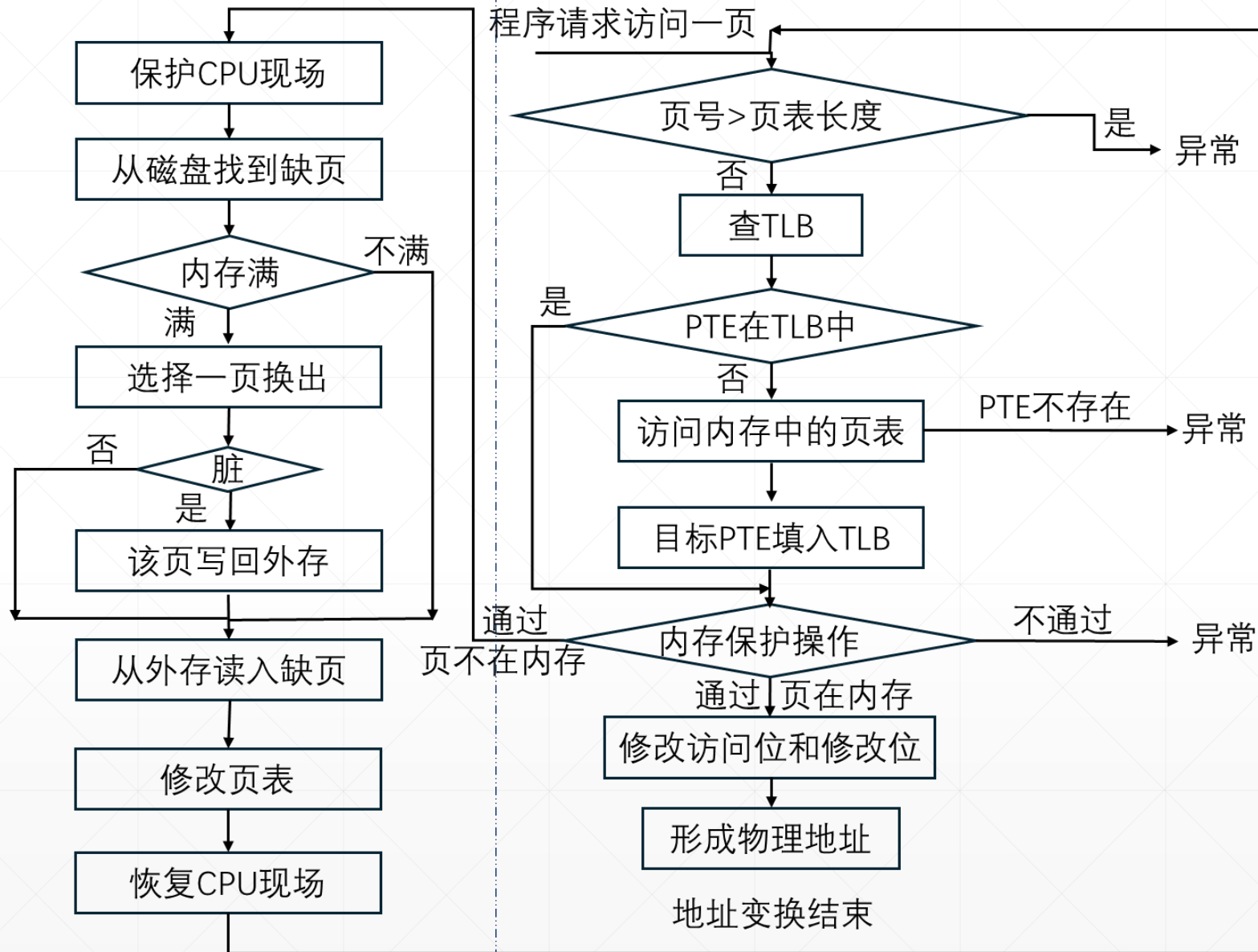
# 5、 请求调页系统概念设计（页表 和 页表项 PTE）

|       | 物理页框号  | 外存地址  | 访问字段 A | 修改位M | 状态位 P | 其余prot属性 |
|-------|--------|-------|--------|------|-------|----------|
|       | .....  |       |        |      |       |          |
| 0x266 | 0x5123 | 0x200 | 1      | 1    | 1     | U RO X   |
| 0x267 | 0x3567 | 0x420 | 无意义    | 无意义  | 0     | U RW ~X  |

PTE的下标是逻辑页号  
比如，虚地址为0x266478  
的指令，逻辑页号是0x266

- (1) 状态位P (Present)：逻辑页是否驻留内存。
  - 1，是。有分配内存，物理页框号有效。0，否。尚未分配内存，物理页框号无效。
- (2) 外存地址：逻辑页在磁盘上的起始扇区号。
  - 1页，连续8个扇区
- (3) 访问字段A (Access)：记录逻辑页面在一段时间内被访问的情况。
  - 可以是访问次数，最近访问时刻 或 仅仅用1个bit表示最近有没有访问过
- (4) 修改位M (Modify)：又叫脏位D (Dirty)。逻辑页面在调入内存后是否被写过。
  - 脏页面，写过的页面，淘汰时需要写回外存。

# 请求调页操作



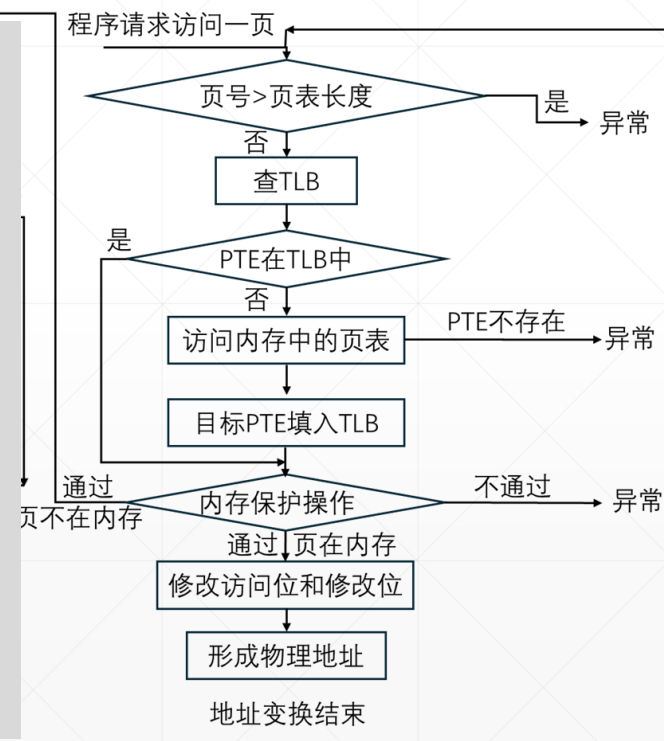
# 细节 MMU硬件逻辑

逻辑地址

|  |  |           |  |  |  |  |                |  |  |
|--|--|-----------|--|--|--|--|----------------|--|--|
|  |  | 页号 (page) |  |  |  |  | 页内偏移量 (offset) |  |  |
|--|--|-----------|--|--|--|--|----------------|--|--|

| 主存页框号  | 外存地址  | 访问字段<br>(access) | 修改位<br>(dirty) | 状态位<br>(present) | U/S R/W<br>& others |
|--------|-------|------------------|----------------|------------------|---------------------|
| .....  |       |                  |                |                  |                     |
| 0x5123 | 0x200 | 1                | 1              | 1                | U RW                |
| 0x3567 | 0x420 | 无意义              | 无意义            | 0                | U RW                |

- 逻辑地址取页号 (page)。
- 用页号查TLB。
  - 若不命中，用页号查主存中的页表。取PTE，填入TLB。
    - PTE==null，抛出缺页异常 (分支1)。
- 使用目标PTE，执行内存保护和 请求调页 操作。
  - 内存保护。
    - 权限检测未通过 (RO、U、X)，抛出缺页异常 (分支2)。
  - 请求调页。用存在位判断逻辑页是否驻留主存。
    - 若P==0，抛出缺页异常 (分支3)、请求调页。此次地址映射结束。
    - 若P==1，地址映射 (生成物理地址 PTE.base 拼 offset)，PTE维护 (修改 prot: 访问位置1、修改位写时置1。TLB、内存同步修改)。

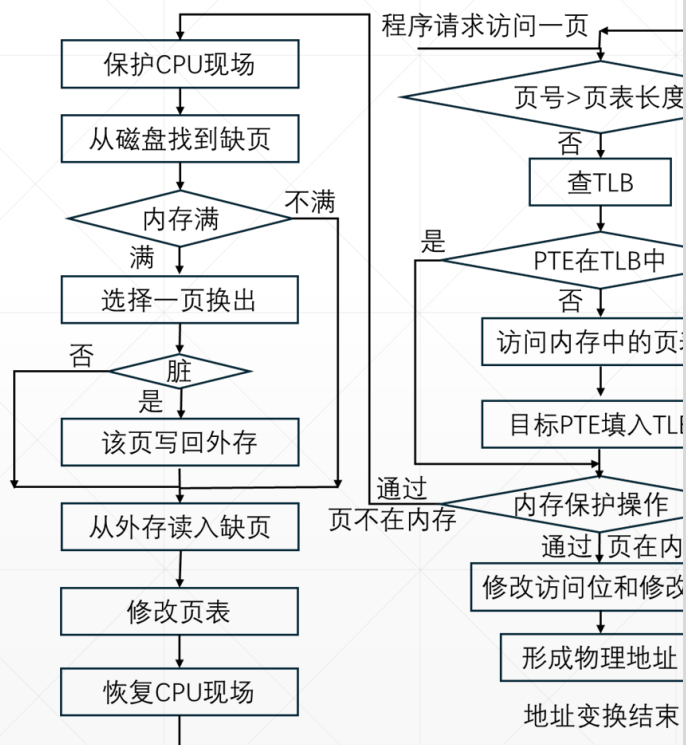


# 缺页异常处理程序，请求调页逻辑

逻辑地址 (cr2)

|  |           |  |  |  |                |  |
|--|-----------|--|--|--|----------------|--|
|  | 页号 (page) |  |  |  | 页内偏移量 (offset) |  |
|--|-----------|--|--|--|----------------|--|

| 主存页框号  | 外存地址  | 访问字段 (access) | 修改位 (dirty) | 状态位 (present) | U/S R/W & others |
|--------|-------|---------------|-------------|---------------|------------------|
| .....  |       |               |             |               |                  |
| 0x5123 | 0x200 | 1             | 1           | 1             | U RW             |
| 0x3567 | 0x420 | 无意义           | 无意义         | 0             | U RW             |



- 1、用页号page查运行进程的页表，定位所缺页面的PTE，获得所缺页面在外存中的地址。
- 2、内存满吗？
  - 不满。分配一个空闲物理页框base，直接调页。
  - 满。执行页面置换算法，淘汰一个逻辑页面，处理它的PTE。
    - 脏？将页面写回盘交换区。会入睡。修改位（脏bit）清0。
    - 修改内存页表。PTE，P清0。无效TLB中的复本。
    - 记下来它的物理页框号base。
- 4、调页。启动IO，读入所缺页面，写入分配给它的物理页框base。处理PTE。P置1；填base；访问位，修改位清0。

缺页异常返回，重新执行引发缺页的那条指令。被访问逻辑页在内存，不会再缺页。

## 例题

|       | 物理页框号  | 外存地址  | 访问字段 A | 修改位M | 状态位 P | 其余prot属性 |
|-------|--------|-------|--------|------|-------|----------|
|       | .....  |       |        |      |       |          |
| 0x266 | 0x5123 | 0x200 | 1      | 1    | 1     | U RO X   |
| 0x267 | 0x3567 | 0x420 | 无意义    | 无意义  | 0     | U RW ~X  |

将线性地址翻译成物理地址。写出详细的地址映射过程。

- 1、线性地址0x266478，读操作
- 2、线性地址0x266478，写操作
- 3、线性地址0x267100，写操作

|       | 物理页框号  | 外存地址  | 访问字段 A | 修改位M | 状态位 P | 其余prot属性 |
|-------|--------|-------|--------|------|-------|----------|
|       | .....  |       |        |      |       |          |
| 0x266 | 0x5123 | 0x200 | 1      | 1    | 1     | U RO X   |
| 0x267 | 0x3567 | 0x420 | 无意义    | 无意义  | 0     | U RW ~X  |

### 1、线性地址0x266478，读操作

- 页号：0x266。 PTE存在 且 P bit是1，主存命中。
- 内存保护操作：
  - RO，读操作合法
- PTE.access = 1;
- 生成物理地址： 0x5123478

### 2、线性地址0x266478，写操作

- 页号：0x266。 PTE存在 且 P bit是1，主存命中。
- 内存保护操作：
  - RO，写操作非法。抛出14#缺页异常。

14#异常处理程序：写RO页面，prot不对，这是非法内存访问，向现运行进程发信号SIGSEGV，杀死出错进程，阻止应用程序的非法内存访问。



### 3、线性地址0x267100，写操作（有空闲物理页框）

- 页号：0x267。PTE存在。
- 内存保护操作。
  - PTE, RW bit是1。写操作合法
- 请求调页操作
  - PTE, Present bit是0，抛出缺页异常，引发缺页的线性地址 0x267100 → cr2
- 执行缺页异常处理程序
  - 为0x267#逻辑页面（cr2 去掉末尾 100）分配一个物理页框，0x5555。
  - 假设0x5555空闲，分配给0x267#逻辑页。
  - 调页：向磁盘发IO命令、之后，进程入睡放弃CPU。IO参数如下：
    - 内存首地址：0x5555000
    - 磁盘起始扇区：0x420（0x267#逻辑页面在磁盘上的首地址）
    - 数据传输量：4096字节
    - 读操作（扇区0x420，0x421，0x422，...，0x427 → 0x5555页框）
  - 修改0x267#PTE：主存页框号=0x5555，P=1，修改位 = 0，访问字段 = 0
  - 缺页异常返回，重新执行访问线性地址0x267100的指令
- 页号：0x267。PTE present bit为1，进程拥有本页面的写权限。
  - 修改位 = 1，访问字段 = 1
  - 生成物理地址：0x5555100

|       | 主存页框号  | 外存地址  | 访问字段<br>(access) | 修改位<br>(dirty) | 状态位<br>(present) | U/S R/W<br>& others |
|-------|--------|-------|------------------|----------------|------------------|---------------------|
|       | .....  |       |                  |                |                  |                     |
| 0x266 | 0x5123 | 0x200 | 1                | 0              | 1                | U RW                |
| 0x267 | 0x3567 | 0x420 | 无意义              | 无意义            | 0                | U RW                |

IO完成，磁盘中断处理程序唤醒本进程

异常处理程序继续执行

|       | 主存页框号  | 外存地址  | 访问字段<br>(access) | 修改位<br>(dirty) | 状态位<br>(present) | U/S R/W<br>& others |
|-------|--------|-------|------------------|----------------|------------------|---------------------|
|       | .....  |       |                  |                |                  |                     |
| 0x266 | 0x5123 | 0x200 | 1                | 0              | 1                | U RO                |
| 0x267 | 0x5555 | 0x420 | 1                | 1              | 1                | U RW                |



### 线性地址0x267100，写操作

- 页号：0x267。 PTE存在。
- 内存保护操作。
  - PTE, RW bit是1。写操作合法
- 请求调页操作
  - PTE, Present bit是0，抛出缺页异常，线性地址 0x267100 → cr2
- 执行缺页异常处理程序
  - 为0x267#逻辑页面（cr2 去掉末尾 100）分配一个物理页框，0x5555。
  - 假设**0x5555脏**。
- 调页：向磁盘发IO命令、之后，进程入睡放弃CPU。IO参数如下：
  - 内存首地址：0x5555000
  - 磁盘起始扇区：0x420（0x267#逻辑页面在磁盘上的首地址）
  - 数据传输量：4096字节
  - 读操作（扇区0x420, 0x421, 0x422, ..., 0x427 → 0x5555页框）
- 修改0x267#PTE：主存页框号=0x5555, P=1, 修改位 = 0, 访问字段 = 0
- 缺页异常返回，重新执行访问线性地址0x267100的指令
- 页号：0x267。PTE present bit为1，进程拥有本页面的写权限。
  - 修改位 = 1, 访问字段 = 1
  - 生成物理地址：0x5555100

|       | 主存页框号  | 外存地址  | 访问字段<br>( access ) | 修改位<br>( dirty ) | 状态位<br>( present ) | U/S R/W<br>& others |
|-------|--------|-------|--------------------|------------------|--------------------|---------------------|
|       | .....  |       |                    |                  |                    |                     |
| 0x266 | 0x5123 | 0x200 | 1                  | 0                | 1                  | U RW                |
| 0x267 | 0x3567 | 0x420 | 无意义                | 无意义              | 0                  | U RW                |

0x5555#物理页框写回盘交换区的过程：

- 1、若PTE外存地址为空，盘交换区分配4096字节（连续的8个扇区），外存地址 = 起始扇区号start。
- 2、0x5555#物理页框写回磁盘，从start开始，连续写8个扇区。
- 3、修改映射至0x5555页框的 PTE：P=0。

## 5、页面置换算法（for 固定分配局部置换）

缺页时，若内存已无空闲物理页框，淘汰一页。用新逻辑页面覆盖被淘汰页面。若被淘汰页面脏（修改过），写回磁盘。

页面置换算法决定淘汰哪一页。

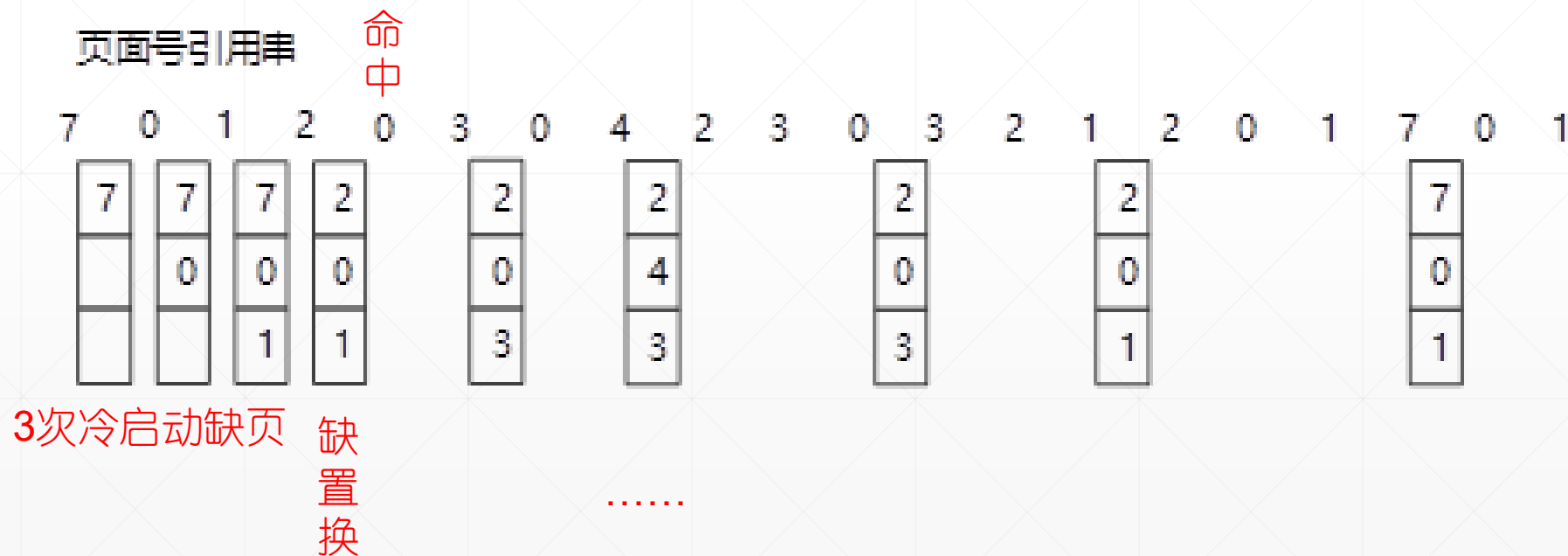
- 1、OPT， 未来访问时刻最远的（最优）
- 2、FIFO， 最先进入内存的
- 3、LRU， 最近最久未访问的（Least Recently Used）
- 4、LRU近似算法，比如clock
- 5、随机， 随便

缺页率  
越低越好

# 最优替换算法 (OPT)

未来访问时刻最远的，被淘汰

理论上的最优算法，可以保证缺页率最低



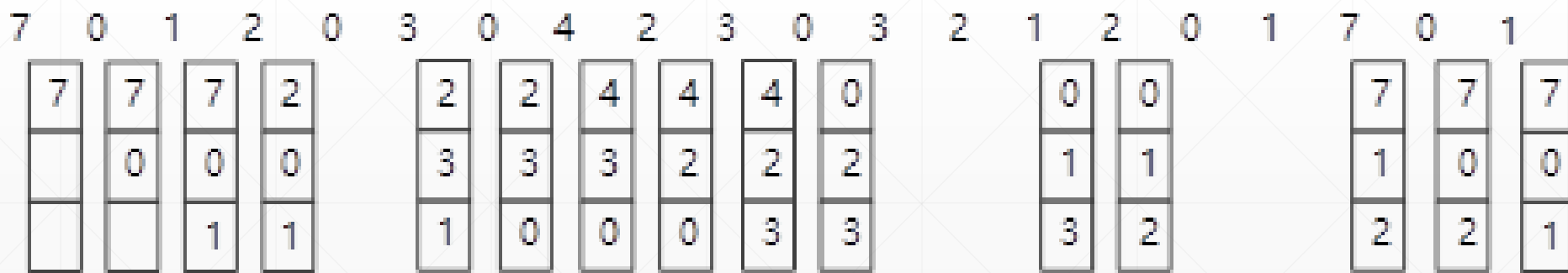
20次内存引用  
9次缺页  
6次页面置换  
缺页率 =  $9/20 = 45\%$

# 先进先出页面置换算法 (FIFO)

最先进入内存的，被淘汰

隐含假设：程序顺序结构

页面号引用串



20次内存引用  
15次缺页  
12次页面置换  
缺页率 =  $15/20 = 75\%$

3次冷启动缺页  
缺置换

# FIFO的性能

- 容易实现，置换算法  $O(1)$ 。
- 缺点：无法保证经常被访问的逻辑页面不被淘汰。这些页面被淘汰后，很可能马上又被访问、被换回内存覆盖其它页面。
- 缺页率/页面置换率高的原因在于，用页面进入内存的先后次序去推测未来访问时刻不合理。最先进入内存的页面，可能会频繁访问，不该被淘汰。

# FIFO算法, Belady异常

- 直观的感觉：如果我们为一个进程分配更多的物理页框（物理块），它的缺页率会下降，会跑得更快。  
但并不总是这样，看反例。下图使用 FIFO页面置换算法。

| 访问页面   | 3 | 2 | 1 | 0 | 3 | 2 | 4 | 3 | 2 | 1 | 0 | 4 |
|--------|---|---|---|---|---|---|---|---|---|---|---|---|
| 物理块 1  | 3 | 3 | 3 | 0 | 0 | 0 | 4 |   |   | 4 | 4 |   |
| 物理块 2  |   | 2 | 2 | 2 | 3 | 3 | 3 |   |   | 1 | 1 |   |
| 物理块 3  |   |   | 1 | 1 | 1 | 2 | 2 |   |   | 2 | 0 |   |
| 缺页否    | √ | √ | √ | √ | √ | √ | √ |   |   | √ | √ |   |
| 物理块 1. | 3 | 3 | 3 | 3 |   |   | 4 | 4 | 4 | 4 | 0 | 0 |
| 物理块 2. |   | 2 | 2 | 2 |   |   | 2 | 3 | 3 | 3 | 3 | 4 |
| 物理块 3. |   |   | 1 | 1 |   |   | 1 | 1 | 2 | 2 | 2 | 2 |
| 物理块 4. |   |   |   | 0 |   |   | 0 | 0 | 0 | 1 | 1 | 1 |
| 缺页否    | √ | √ | √ | √ |   |   | √ | √ | √ | √ | √ | √ |

3个物理块, 9次缺页

4个物理块, 10次缺页

这是极其个别的现象。不必纠结。  
继续追加物理页框，缺页率会迅速下降。

# LRU、LFU: OPT的平替

程序执行完毕之前，无法得知每张页面的下次访问时刻。所以，**OPT**是无法实现的理论算法。只能作为参照，衡量其余算法的性能。

由于程序运行有局部性原理，我们可以用页面 **最近的访问历史** 推测 **未来的访问情况**。

最优替换策略 可行的替代策略：

- 最近经常访问的页面，未来被访问的概率高。最不经常使用的页面应该被淘汰 (**Least Frequently Used, LFU**)
- 很久没访问的页面，未来被访问的概率低。被淘汰的是最近最久未使用的页面 (**Least Recently Used, LRU**)

## ▪ LFU 算法跟踪页面的实际访问情况

- 为进程分配若干物理页框，每个页框引用计数 **count** 初始化为0。
- 每次内存访问，物理页框 **count++**。
- 发生缺页时，用 **count** 值最小的物理页框装所缺页面。清0所有页框的**count**。

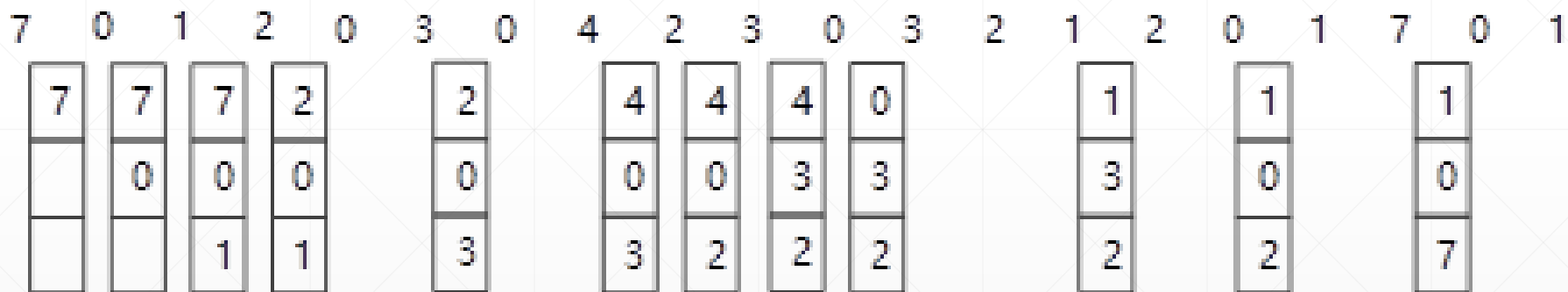
## ▪ LRU 算法跟踪页面的实际访问情况

- 为进程分配若干物理页框，形成LRU链表（双向队列）。
- 每次内存访问，从LRU队列摘除被访问页框，将其插入队尾。
- 缺页时，用队首物理页框装所缺页面。

# 最近最久未使用 (LRU)

淘汰上一次访问时刻最早的

页面号引用串



12次缺页。  
20次内存引用。  
缺页率 =  $12/20 = 60\%$

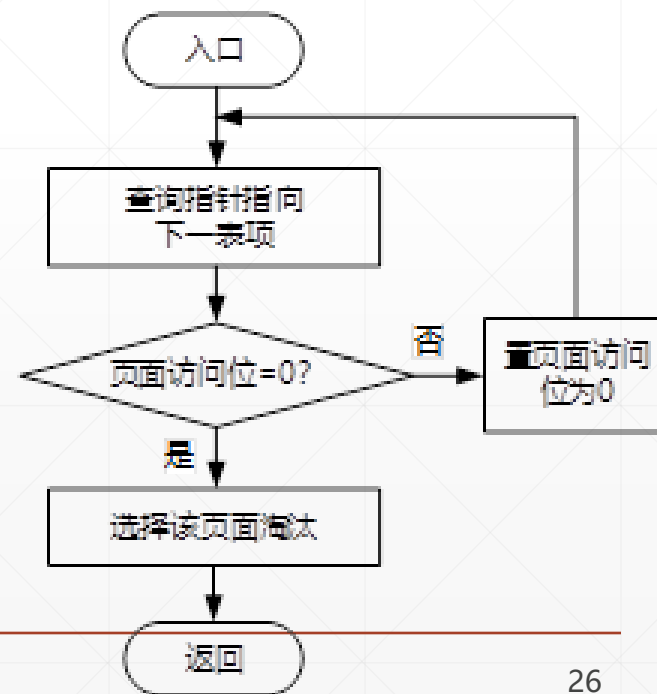
- 算法跟踪页面的实际访问情况，性能好
- 但，每次访存需要维护LRU链表，开销无法忍受

# Clock (NRU, Not Recently Used) 淘汰最近没用过的 (不一定是最久)

- MMU, 地址映射与内存映射逻辑不变。某页被访问时, 硬件将PTE中的访问位 (A bit) 置1。

Clock算法在页表外维护一个循环队列 (队列中的每个元素是一根指针, 指向驻留页面的PTE)。  
一根当前指针 current。

```
while() {  
    if (current→pte.A==1)  
        current→pte.A=0, current= current→next;  
    else {  
        PTE[page].base = current→pte.base, current→pte = PTE[page];  
        current= current→next, break; }  
} // 遍历循环队列, 直至找到访问位A为0的PTE
```



# 例:

为进程分配3个物理页框。系统初始状态:  $current = 0$ , 所有页框 访问位A 清 0  
 \* 表示访问位 1;  $\leftarrow$  标识 $current$ 指针所指位置。

|              |              |              |                  |                |                |                  |                  |                |                  |                  |                  |
|--------------|--------------|--------------|------------------|----------------|----------------|------------------|------------------|----------------|------------------|------------------|------------------|
| 2            | 3            | 2            | 1                | 5              | 2              | 4                | 5                | 3              | 2                | 5                | 2                |
| 2 *          | 2 *          | 2 *          | 2 * $\leftarrow$ | 5 *            | 5 *            | 5 * $\leftarrow$ | 5 * $\leftarrow$ | 3 *            | 3 *              | 3 * $\leftarrow$ | 3 * $\leftarrow$ |
| $\leftarrow$ | 3 *          | 3 *          | 3 *              | 3 $\leftarrow$ | 2 *            | 2 *              | 2 *              | 2 $\leftarrow$ | 2 * $\leftarrow$ | 2 *              | 2 *              |
|              | $\leftarrow$ | $\leftarrow$ | 1*               | 1              | 1 $\leftarrow$ | 4 *              | 4 *              | 4              | 4                | 5 *              | 5 *              |

命中  
clock  
没跑

命中  
NRU  
没跑

命中  
NRU  
没跑

命中  
NRU  
没跑

自己写一遍

|      |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|------|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 访问顺序 | 7 | 0 | 1 | 2 | 0 | 3 | 0 | 4 | 2 | 3 | 0 | 3 | 2 | 1 | 3 | 2 |
| 帧1   | 7 | 1 | 7 | 1 | 7 | 1 | 7 | 1 | 3 | 1 | 3 | 1 | 3 | 1 | 3 | 1 |
| 帧2   |   | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 |
| 帧3   |   |   |   | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 1 | 0 | 4 | 1 | 4 | 1 |
| 帧4   |   |   |   |   | 2 | 1 | 2 | 1 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 1 |
| 缺页否  | √ | √ | √ | √ |   | √ |   | √ |   |   |   |   |   | √ |   | √ |

图 3.26 CLOCK 算法时的置换图

首次访问 7, 0, 1, 2 时, 产生缺页中断, 依次调入主存, 访问位都置为 1。访问 0 时, 已存在, 访问位置为 1。访问 3 时, 产生第 5 次缺页中断, 替换指针初始指向帧 1, 此时所有帧的访问位均为 1, 则替换指针完整地扫描一周, 把所有帧的访问位都置为 0, 然后回到最初的位置 (帧 1), 替换帧 1 中的页 (包括置换页面和置访问位为 1), 如图 3.27(a) 所示。访问 0 时, 已存在, 访问位置为 1。访问 4 时, 产生第 6 次缺页中断, 替换指针指向帧 2 (上次替换位置的下一帧), 帧 2 的访问位为 1, 将其修改为 0, 继续扫描, 帧 3 的访问位为 0, 替换帧 3 中的页, 如图 3.27(b) 所示。然后依次访问 2, 3, 0, 3, 2, 均已存在, 每次访问都将其访问位置为 1。访问 1 时, 产生缺页中断, 替换指针指向帧 4, 此时所有帧的访问位均为 1, 又完整扫描一周并置访问位为 0, 回到帧 4, 替换之。访问 3 时, 已存在, 访问位置为 1。访问 2 时, 产生缺页中断, 替换指针指向帧 1, 帧 1 的访问位为 1, 将其修改为 0, 继续扫描, 帧 2 的访问位为 0, 替换帧 2 中的页。

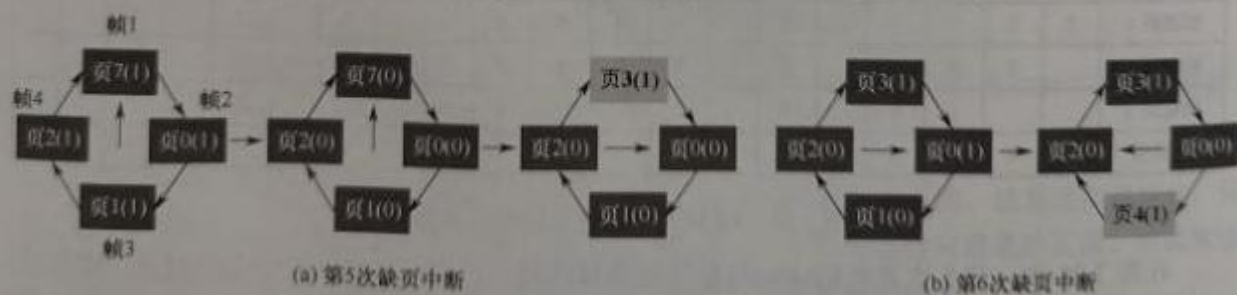


图 3.27 缺页中断时替换指针扫描示意图

# 改进型Clock置换算法 —— 优先淘汰干净的页面

访问位A和修改位(脏位)D的4种组合。描述了页框的四种不同的类型。

- 1类:  $A=0, D=0$  最近没有被访问, 也没有被修改;
- 2类:  $A=0, D=1$  最近没有被访问, 但被修改;
- 3类:  $A=1, D=0$  最近被访问过, 但没有被修改;
- 4类:  $A=1, D=1$  最近被访问过, 被修改;

淘汰这四种页面, 代价?

# 改进型Clock置换算法的实现

显然第1类页面是最佳淘汰页面，应尽量避免淘汰第4类页面。算法执行的步骤如下：

- 1 从current指针开始扫描内存页框，寻找第1类页面（未访问过的干净页面）予以替换。
- 2 如果第1步操作失败，重新扫描，寻找第2类页面（未访问过的脏页面），对扫描过程中途径的所有页帧A位清0。
- 3 如果第2步也失败，指针将回到原来的位置（所有页帧A位为0）。重复第1步，必要时重复第2步。一定能找到淘汰页（最近被访问过的页面）。



# 不考虑页面访问历史的置换算法

随机  
FIFO



# 请求调页虚拟存储器相关的理论——工作集和驻留集

- **进程  $P_i$  的工作集：**未来一小段时间内，需要使用的所有逻辑页面。工作集大小  $N_i$ 。
- **进程的驻留集：**当前存储在内存中的所有逻辑页面
- **工作集是驻留集的子集，不会缺页**
- **当应用程序从一个计算阶段过渡到下个计算阶段时，工作集会发生变化。这个阶段进程会缺页，持续一段时间。之后，进程稳定工作于下个计算阶段，不再会经常缺页。**
- **如果一个进程频繁缺页，就该为它追加主存页框。**

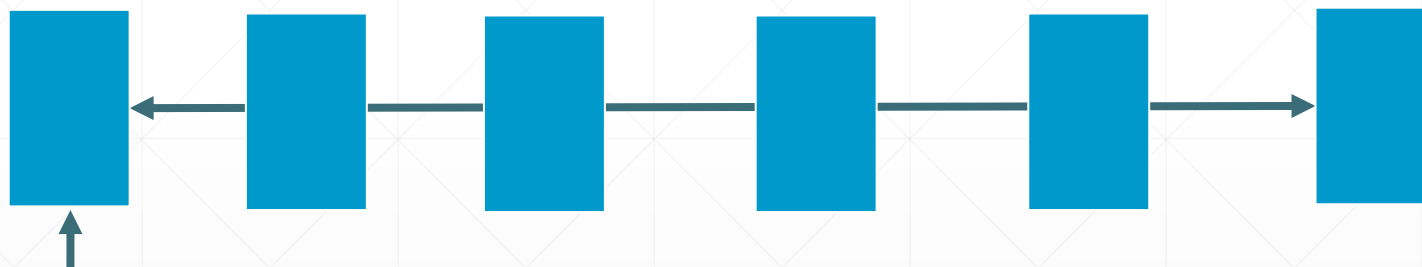


## 请求调页虚拟存储器相关的理论——抖动

- 如果系统接纳了过多的作业，就会出现主存物理页框不足，无法容纳任意进程工作集的情况，此时所有进程均缺页频繁。
- $\sum N_i \gg$  内存总容量，系统抖动。主机和主硬盘忙于缺页异常任务，无法执行有效计算。
- 抖动的解决方法：将部分进程挂起，驻留集全部交换至磁盘，空出来的物理页框分配给留在内存的其它进程。保证内存中的进程可以顺利运行。

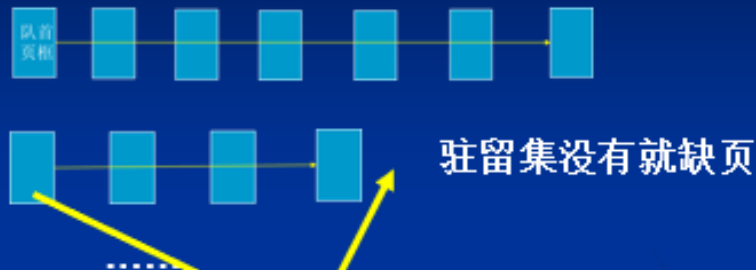
# 置换算法的实现

- 1、固定分配局部替换 FIFO

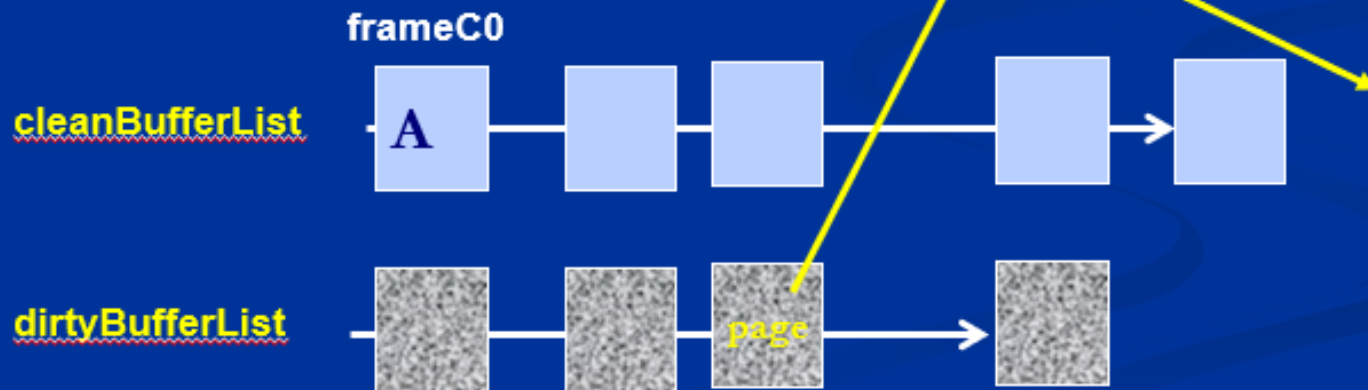


## 2、页面缓存算法（空闲页框池里有所缺页面）

分配给每个进程固定数量的页面就是驻留集了



系统维护的空闲页框池



每个进程，一条**FIFO**队列。进程访问的逻辑页面**page** **FIFO**队列不存在，缺页。

系统维护空闲页框池。其中的物理页框所有进程共用。

空闲页框池有两条**LRU**队列：干净缓存队列和脏缓存队列。

缺页时：

1、**page**，空闲页框池里有吗？

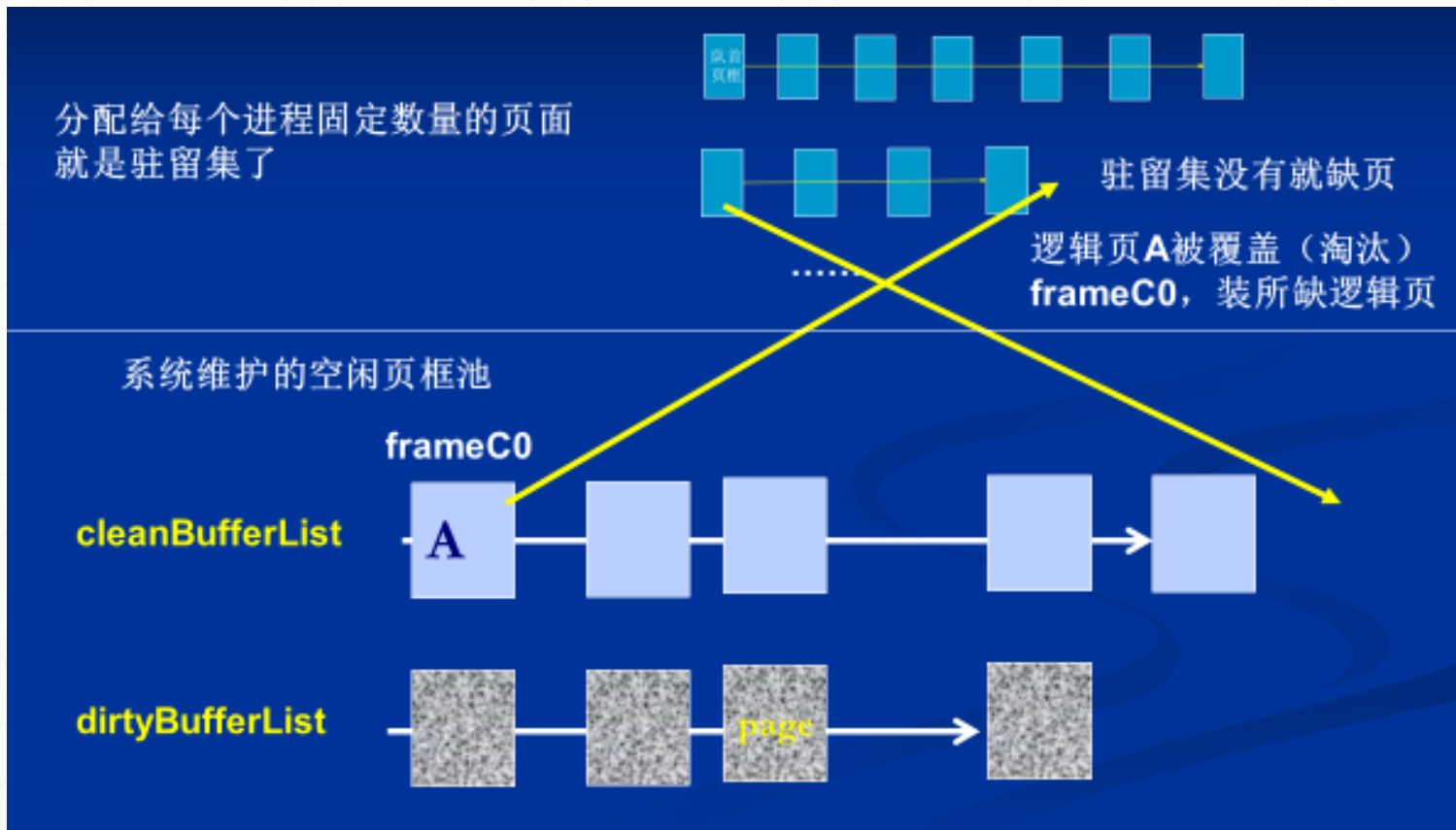
Y，摘出来，挂进程**FIFO**队尾。

N，用**frameC0**装所缺页面，原先里面的逻辑页**A**被淘汰（里面装的是谁的页面？不知道，不要紧）。

**frameC0**挂队尾

2、摘**FIFO**队列队首，挂空闲页框池的队尾。

# 页面缓存算法（空闲页框池里没有所缺页面）



- 1、page，空闲页框池里有吗？  
Y，摘出来，挂队尾。  
N，用frameC0装所缺页面，原先里面的逻辑页A被淘汰（里面装的是谁的页面？不知道，不要紧）。  
frameC0挂FIFO队列队尾
- 2、摘FIFO队列队首，挂空闲页框池的队尾。